TITLE:

MOBILE APP DEVELOPMENT PROCESS

CEF 440, Internet programming and Mobile Programming

Lecturer: Dr Valery Nkemeni

Report by Group 21

S/N	NAME	Specialisation	Matricule
1	CHE RANDY FANG	Software	FE23A030
2	ECHUNDIP CARL-LAWRENCE N.	Networking	FE23A040
3	MAKANE THANKS MOSAKA	Software	FE23A085
4	NUMFOR PRECIOUS LUM	Software	FE23A126
5	TEMFA NKOMBONG FORTUNEE	Software	FE23A168

Table of contents

1. Introduction……3
   - 1.1 What are mobile applications
   - 1.2 Why are they important.
   - 1.3 Mobile applications over websites?
2. Types of mobile applications……4
   - 2.1 Native mobile applications
   - 2.2 Web applications
   - 2.3 Hybrid applications
   - 2.4 The Big Question...
3. Mobile App Programming Languages……5
   - 3.1 Varieties
   - 3.2 What to take note of
4. Mobile App development frameworks……6
   - 4.1 introduction
   - 4.2 Importance of frameworks
   - 4.3 Key frameworks for mobile development
   - 4.4 Comparism and selection
5. Mobile app architecture and design patterns……8
   - 5.1 Definition
   - 5.2 Various types and comparisons
6. Requirement Engineering process…….9
   - 6.1 Stakeholder Identification and classification
   - 6.2 Problem definition and scope framing
   - 6.3 Requirement extraction phase
   - 6.4 Documentation
7. Mobile app development cost estimation……..10
   - 7.1 Cost drivers
   - 7.2 Estimation techniques
   - 7.3 Hidden costs
8. Conclusion……..11
9. References……...11

1. Introduction:

1.1 ***What are mobile applications***

Mobile applications (apps) are software programs designed to run on smartphones and tablets such as Android and iOS devices. Today, they are a big part of everyday life and some businesses or organisations cannot function without them as they may be the only tools used for communication or even as education and entertainment tools.

1.2 Why are they important:

With the rapid growth of smartphones, mobile apps have become essential tool that help people solve problems, stay connected and access services easily. For developers and Businesses, mobile apps also create opportunities to innovate, reach users globally and improve efficiency.

1.3. Mobile applications over websites?

Over a long time of usage, we come to understand the advantages Mobile Applications have over Web applications:

- ✓ Better performance and responsiveness: Native apps run compiled or optimized code for the device/OS, so they generally deliver faster load times, smoother animations, and lower latency than web apps.
- ✓ Deeper access to device hardware and OS features: Direct access to sensors and features: camera, GPS, accelerometer/gyroscope, Bluetooth, NFC, microphone, biometric authentication (fingerprint/Face ID), push notifications, contacts, calendar, calendars, and device storage.
- ✓ Offline functionality and local storage: Native apps can store data locally and operate fully or partially offline, syncing when connectivity returns.
- ✓ Native UI/UX and platform conventions: Apps can use platform-specific UI components and follow OS interaction patterns, producing consistent, intuitive experiences for users.
- ✓ Improved security and sandboxing (help isolates one app from another through out the operating system): Mobile OSs provide app sandboxing, secure key stores, and platform-managed permission controls.

2. Types of Applications:

2.1 ***Native Applications:***

Native apps are built specifically for one platform, either Android or iOS.
For Android, developers often use tools like Android Studio, while iOS apps are built using Xcode.
They offer the following functionalities:
High performance, Smooth user experience and have Full access to device features
Example: WhatsApp.

2.2 ***Web Applications:***

Web apps run in mobile browsers and are built using web technologies like HTML, CSS, and JavaScript.
They have the following characteristics:
Do not need installation, Work across devices and are easy to maintain
Example: Gmail (browser version).

2.3 ***Hybrid Applications***

Hybrid apps combine both native and web technologies. They are built once and can run on multiple platforms.
They:
Reduce development time, are cost-effective and offer moderate performance
Example: Instagram (partially hybrid).

2.4 The big Question

Usually the question "Why does this company have both a mobile app and a website at the same time pops up"
We can answer this by telling how advantageous it is in the industry.

1. Easy access anytime, anywhere:
Websites can be accessed on any device through a browser, while mobile apps are always available on your phone.
2. Better user experience:
Mobile apps are faster and smoother, while websites are great for quick access without installing anything.
3. Offline access (Mobile Apps)
Apps can work even without internet for some features, unlike websites.
4. Real-Time notifications
Mobile apps can send instant updates (notifications), helping users stay informed always.
5. Easy discovery
Websites help users find services easily through search engines, while apps provide a more personalized experience once installed.

3. Mobile App Programming Languages:

Mobile application development usually relies heavily on programming languages, and these languages vary depending on the platform being targeted. Choosing the right programming language is important because it affects the performance, scalability, and overall user experience of the application. Today, mobile developers mainly focus on Android, iOS, and cross-platform development, each with its own preferred languages.

3.1. Varieties

Android Development:
Android applications are traditionally developed using **Java**, one of the most widely used programming languages in the world. Java is known for its stability, large community support, and strong performance. However, in recent years, **Kotlin** has become the modern standard for Android development.
Kotlin is now widely preferred because it is simpler and more concise, allowing developers to write less code while achieving the same functionality. It also reduces common programming errors, making applications more reliable and easier to maintain.

iOS Development
For iOS applications, developers primarily use **Swift** and **Objective-C**. Objective-C is the older language that was used for many years to build Apple applications and although it is still in use today, it is gradually being replaced by Swift.
Swift is now the main programming language for iOS development because it offers several advantages. It is faster in performance, easier to read and write, and provides better security features. Swift's modern syntax makes it more beginner-friendly and reduces the chances of coding errors.

Cross-Platform Development
Cross-platform development allows developers to build applications that work on both Android and iOS using a single codebase. This approach saves time, reduces development costs, and ensures consistency across platforms.
Popular languages used in cross-platform development include **JavaScript** and **Dart**. JavaScript is commonly used with frameworks like React Native, which enables developers to create mobile apps using web development skills. On the other hand, Dart is used with Flutter, a powerful framework developed by Google that allows for fast and visually appealing application development.

3.2 *What to take note of*:

Mobile app programming languages play a crucial role in the development process. While Java and Kotlin dominate Android development, Swift leads in iOS development. At the same time, cross-platform languages like JavaScript and Dart provide flexible and cost-effective solutions. Understanding these languages helps developers choose the best tools for building efficient, scalable, and user-friendly mobile applications.

4. Mobile App Development Frameworks:

4.1. Introduction:

Mobile app development frameworks are important tools that help developers build applications faster and more efficiently. Instead of writing every part of an app from scratch, these frameworks provide ready-made components, libraries, and basic structures. This reduces repetition, saves time, and helps maintain consistency across different projects. Because of this, developers can focus more on the unique features of their apps rather than the basic setup.

4.2. Importance Of Frameworks

Frameworks are also very useful when developing apps for multiple platforms like Android and iOS. Many modern frameworks allow developers to use a single codebase to build apps for different devices, which makes the whole development process easier and more cost-effective.

4.3. *Key Frameworks for Mobile Development*

Flutter

Primary Language: Dart

Platforms: Android, iOS

Flutter is a framework developed by Google that allows developers to create applications for both Android and iOS using one codebase. One of its main advantages is its speed, especially with features like "hot reload," which allows developers to see changes instantly while working on the app. Flutter also provides a rich set of widgets that make it easier to design attractive and smooth user interfaces. Because of this, it is often used for apps that require good design and high performance.

React Native

Primary Language: JavaScript

Platforms: Android, iOS

React Native is another popular framework developed by Facebook. It allows developers to build mobile apps using JavaScript, which is already widely used in web development. One key benefit is that developers can reuse components, which helps reduce development time. It also has a large community, making it easier to find support and tools. React Native is a good choice for developers who are already familiar with JavaScript.

Xamarin

Primary Language: C#

Platforms: Android, iOS

Xamarin is a framework backed by Microsoft and is mainly used with the C# programming language. It allows developers to create cross-platform apps with performance that is close to native apps. Xamarin integrates well with Microsoft tools like Visual Studio, making it a strong option for enterprise or business applications. It is commonly used in organizations that already rely on Microsoft technologies.

Ionic

Primary Language: HTML, CSS, JavaScript

Platforms: Android, iOS

Ionic is a framework that uses web technologies to build mobile applications. It is especially useful for web developers who want to move into mobile development without learning a completely new language. Ionic makes it easy to create simple and

functional apps quickly, although it may not perform as well as fully native apps in more complex situations.

4.4. Comparison and Selection

Each framework has its own strengths, and the choice depends on the type of app being developed. Flutter is known for its speed and strong user interface design. React Native is popular because of its flexibility and large community support. Xamarin is suitable for enterprise-level applications, while Ionic is ideal for simple apps or quick development.

In general, developers choose a framework based on their experience, the performance required, and the platforms they want to support. As mobile technology continues to grow, frameworks like Flutter and React Native are becoming more widely used because they make development faster and more scalable.

5. Mobile App Architectures and Design Patterns:

5.1 Definition:

Mobile app architecture refers to the way developers organize code into structured components in order to make applications maintainable, scalable, testable, and easy to update. It answers important questions such as where the user interface logic should reside, where business logic is handled, and how data is managed within the application.

5.2 Types of architectures

MVC, MVP, CLEAN, MVVM architectures

One of the most basic architectural patterns is the Model-View-Controller (MVC). In this architecture, the Model is responsible for handling data and business logic, the View is responsible for displaying the user interface, and the Controller manages user input while updating both the Model and the View. The workflow typically involves the user interacting with the View, after which the Controller processes the input, updates the Model, and then refreshes the View with new data. MVC is simple and easy to understand, making it suitable for small applications, prototypes, and beginner projects. However, it has notable drawbacks, such as the tendency for Controllers to become too large (often referred to as the "Massive View Controller" problem), and it becomes difficult to scale for larger applications.

For Clean Architecture, The main layers include the Presentation layer (which contains UI patterns like MVC or MVVM), the Domain layer (which contains core business logic), and the Data layer (which handles APIs and databases). A key principle of Clean Architecture is that dependencies flow inward, meaning that the user interface depends on the domain, but the domain does not depend on the UI. This approach ensures that the system remains highly scalable, maintainable, and testable, and independent of specific frameworks. However, it introduces complexity, requires more boilerplate code, and may be excessive for smaller projects. It is best suited for large applications, enterprise systems, and long-term projects.

When comparing MVC, MVVM, and Clean Architecture, several differences emerge. MVC is low in complexity and best suited for small applications but offers only basic separation and low testability. MVVM provides a medium level of complexity with good separation of concerns and high testability, making it ideal for medium-sized applications. Clean Architecture, on the other hand, has high complexity but offers excellent separation, very high testability, and strong scalability, making it suitable for large-scale applications.

The choice of architecture depends largely on context. MVC is typically chosen for simple applications, quick prototypes, or when learning, as it requires minimal setup and allows for rapid development. MVVM is preferred when building real-world applications with complex user interfaces and a need for testable and maintainable code. Clean Architecture is chosen for large-scale or long-term projects, especially when working in teams and anticipating future scalability, as it allows for independent layers and easier replacement of components such as APIs, databases, or user interfaces.

6. Requirement Engineering Process

Requirement Engineering is a systematic and structured process of identifying, analysing, documenting, validating, and managing the requirements of a software system to ensure that it meets stakeholder needs and operates within defined constraints.

The requirement engineering process is divided into several key phases to ensure all system requirements are properly handled.

a. **Stakeholder identification and classification:**

The objective is to identify everyone who influences or is affected by the system. There are different categories of stakeholders.

- Primary stakeholders: These are the end users if the product i.e. customers, students, staffs, etc. They directly interact with the app
- Secondary stakeholders: These includes the admins, managers and support staffs of the system.
- Tertiary Stakeholders: These are the developers, testers and regulatory bodies of the system.

b. **Problem definition and scope framing:**

In this phase, the problem being solved is defined properly.

The key questions to be answered here are:

What is the current problem? Who experiences it? Why does it matter?

What is included or excluded (the scope of the project)? What is the possible risk?

c. **Requirement extraction phase:**

This is the phase where raw requirements are extracted from stakeholders. This is done using multiple techniques such as;

- Interviews: It is the most powerful technique which involves one-on-one deep discussions and asking of open-ended questions from primary and secondary stakeholders.
- Surveys/ Questionnaires: Coming up with a set of questions to be answered by multiple users so as to collect data and use for patterns.
- Observation: watch users perform tasks and noting hidden problems that is intended to be solved
- Brainstorming Sessions: Having group sessions where ideas are generated. This is useful during the early stages of production.
- Document analysis: Study existing systems, reports and workflows.
- Reverse engineering: Analysing similar apps, finding out what differentiate the app to be build from the others and why it should be used instead.

Sometimes users don't know what they want which may make requirement gathering vague and unrealistic.

d. **Documentation:**

Converting raw input into organized, readable format in an SRS (structured requirement specifications) documents which includes what the project is about the functional requirements (what the system does) non-functional requirements (how the system performs), system features and constraints.

e. **Requirement validation:**

Ensures stakeholders agree with the requirements. They check if the requirements are clear, testable and realistic. It can be done using review meetings, prototyping, use case validation.

The requirements engineering phase defines what to build and prevent ls wasted efforts.

7. Mobile App Development Cost Estimation:

It provides a detailed estimation of the cost required to develop a mobile application, identifying key cost drivers, estimation techniques, sample calculations, hidden costs, and strategic insights. The purpose is to answer the question: "How much will this project cost and why?"

i. **Cost drivers.**

a. App complexity

The complexity of an application is the primary determinant of development cost. More complex apps require advanced architecture, more features, and robust testing, increasing both time and cost.

b. Platform choice

The target platform influences cost due to development and testing requirements. Single platforms (Android or IOS only) have a lower cost, Hybrid platform (flutter / react native) have a moderate cost meanwhile implementing both native has a high cost.

c. Features

Each feature requires development effort and testing. Common features include:

- ✓ User authentication / login system
- ✓ Chat / messaging system
- ✓ Payment integration
- ✓ GPS / location tracking

More features = more time = higher cost.

d. Team Structure

Increasing team size accelerates development but increases cost. Specialized roles (e.g., security engineers) may further raise the budget. A standard development team includes frontend developers, backend developers, UI/UX designer, tester/QA.

e. Development time: Time is directly proportional to cost.

ii. **Estimation Techniques**

a. Expert Judgment: Consult experienced developers or project managers. It is fast, based on real-world experience but subjective and may vary depending on expertise.

b. Analogous Estimation: Compare the project to similar apps previously developed. It is quick and simple but less accurate if project requirements differ significantly.

c. Bottom-Up Estimation: Break the project into smaller components, estimate time and cost for each, then sum up.

iii. **Hidden Costs (Often Overlooked)**

Many indirect costs significantly impact the total project budget:

- Maintenance: Regular updates, bug fixes.
- Server Hosting: Cloud servers, APIs, and databases.
- App Store: Fees registration, transactions, and publishing fees.
- Future Updates: Adding features post-launch based on user feedback.

8. Conclusion:

The above work will help us understand that:

Mobile app development is a structured process that involves planning, designing, developing, and deploying applications. From choosing the right type of application development tools to selecting frameworks and managing costs, each step plays an important role in building successful mobile applications.

9. References:

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill.

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

Griffiths, D., & Griffiths, D. (2019). Head first Android development (3rd ed.). O'Reilly Media.

Holzner, S. (2020). React Native for beginners. McGraw-Hill.

Cursa. (n.d.). Mobile App Architecture Basics: Choosing Patterns that Scale (MVC, MVVM, Clean, and More)

SoftwareHouse. (2025). Using MVVM Architecture in Apps: A Guide to Clean and Scalable Code.

Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Pearson.

Falcon, E. (2025). Mobile App Architecture Patterns: MVC, MVVM, Clean, Redux for Android & iOS Apps. Amazon KDP